JAIN (DEEMED-TO-BE UNIVERSITY)
Faculty of Engineering and Technology, Bengaluru

AWS CLOUD & DATA ENGINEERING

STUDENT NAME	**Twinkle Meska**
USN (ROLL NUMBER)	23BTRSN051
COLLEGE NAME	Jain (Deemed-to-be University) Faculty of Engineering and Technology, Bengaluru
BATCH	2023 – 2027
DATE	12-10-2026
TITLE	AWS CLOUD & DATA ENGINEERING

Glue ETL Pipeline

The diagram below summarises the AWS Glue ETL pipeline followed in this practical: an IAM role is created first, raw CSV data is stored in Amazon S3, a Glue crawler catalogues it, a visual ETL job transforms it, and the cleaned output is written back to S3 in Parquet format.

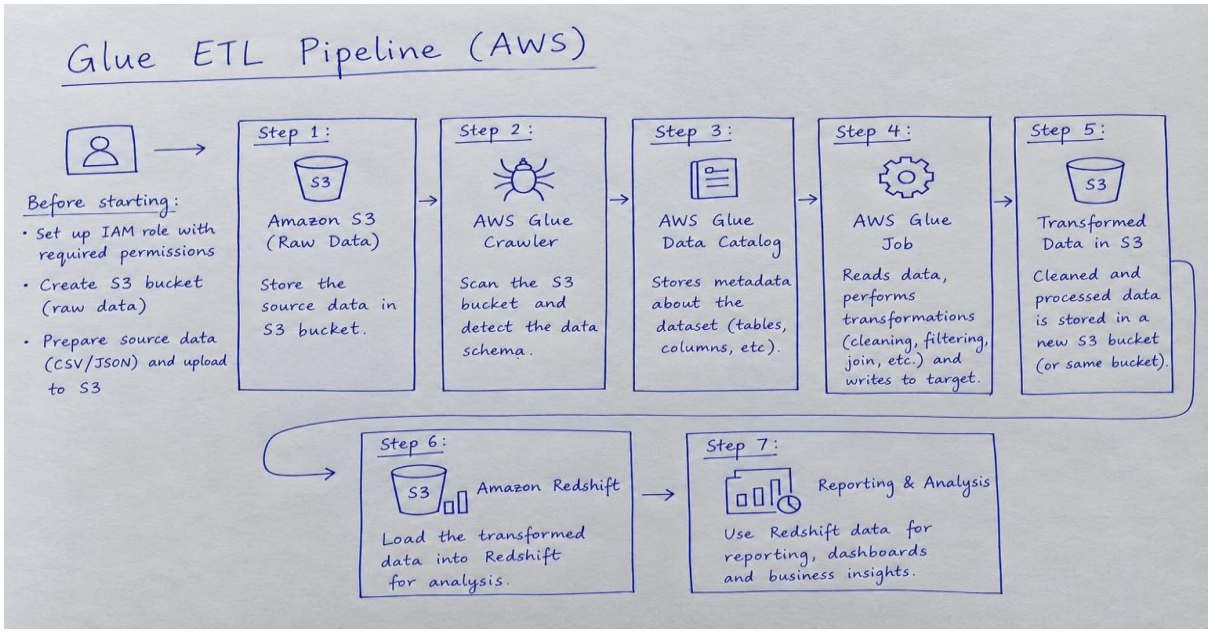


Figure: AWS Glue ETL pipeline (S3 raw → Crawler → Data Catalog → Visual ETL → Transform → S3 curated Parquet).

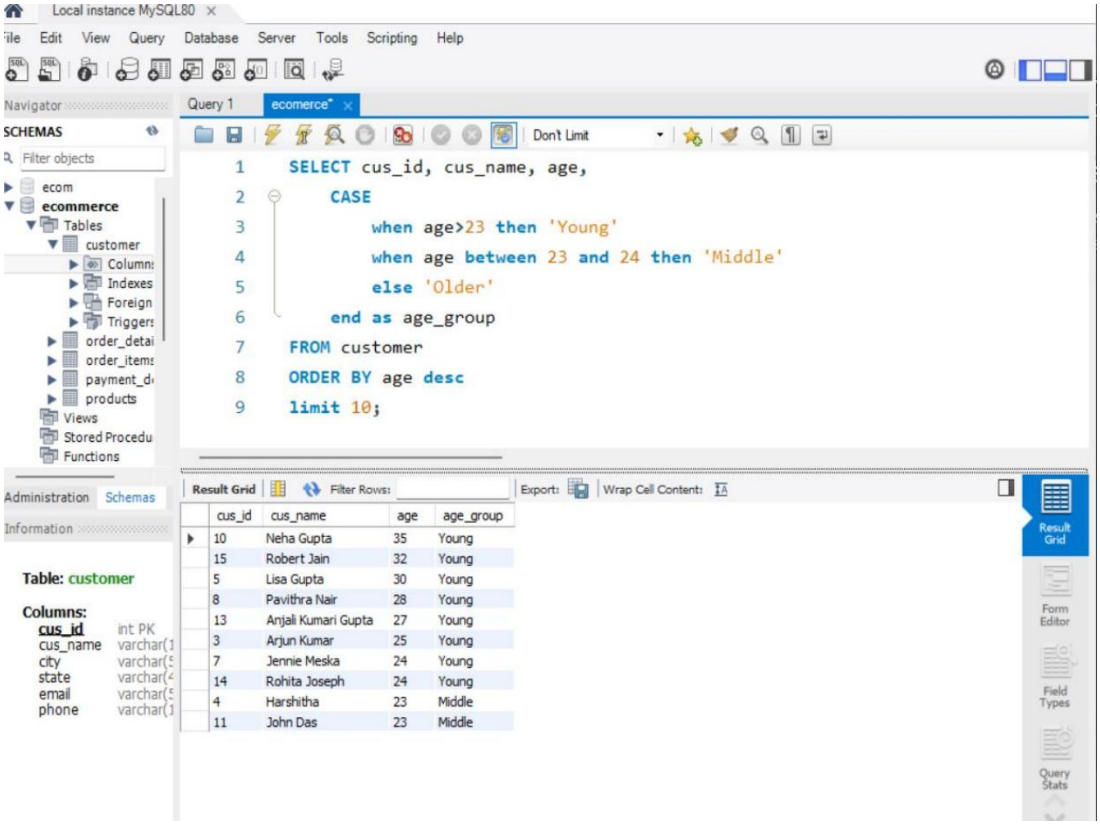
SQL Practice

SQL (Structured Query Language) is the standard language used to create, store, retrieve, update and delete data in a relational database. The queries in this section were written and executed in MySQL Workbench on a local MySQL 8.0 instance, using a database named ecommerce.

The ecommerce database contains five tables: customer, order_details, order_items, payment_details and products. The 15 queries below cover filtering, sorting, grouping, joins, subqueries, CASE expressions, common table expressions and window functions. Each query is shown together with its output.

SQL feature	Use
SELECT / WHERE	Choose columns and filter rows by a condition
ORDER BY / LIMIT	Sort the result and restrict the number of rows
GROUP BY / HAVING	Group rows and filter the groups
SUM() / COUNT() / AVG()	Aggregate functions for totals, counts and averages
CASE	Apply if-else logic inside a query
JOIN / LEFT JOIN	Combine rows from two or more tables
Subquery / WITH (CTE)	Use one query inside another
RANK() / DENSE_RANK() OVER()	Window functions that rank rows

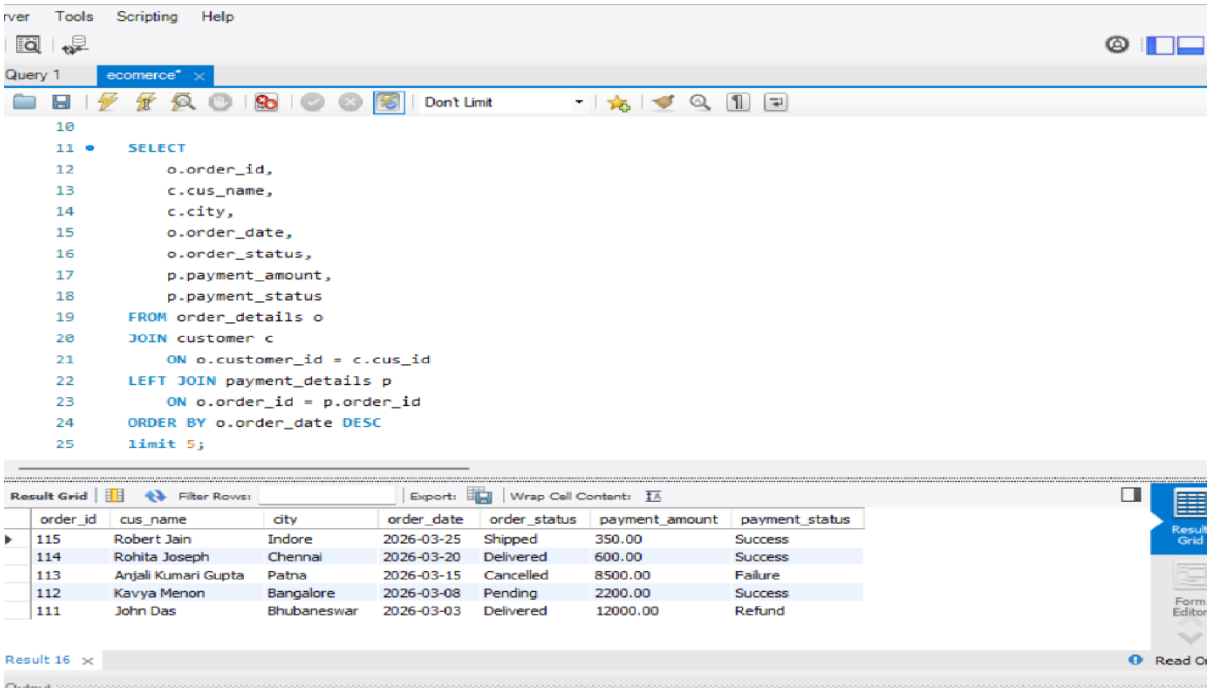
SQL Query 01 — Categorize customers by age using CASE



```
SELECT cus_id, cus_name, age,
    CASE
        when age>23 then 'Young'
        when age between 23 and 24 then 'Middle'
        else 'Older'
    end as age_group
FROM customer
ORDER BY age desc
limit 10;
```

cus_id	cus_name	age	age_group
10	Neha Gupta	35	Young
15	Robert Jain	32	Young
5	Lisa Gupta	30	Young
8	Pavithra Nair	28	Young
13	Anjali Kumari Gupta	27	Young
3	Arjun Kumar	25	Young
7	Jennie Meska	24	Young
14	Rohita Joseph	24	Young
4	Harshitha	23	Middle
11	John Das	23	Middle

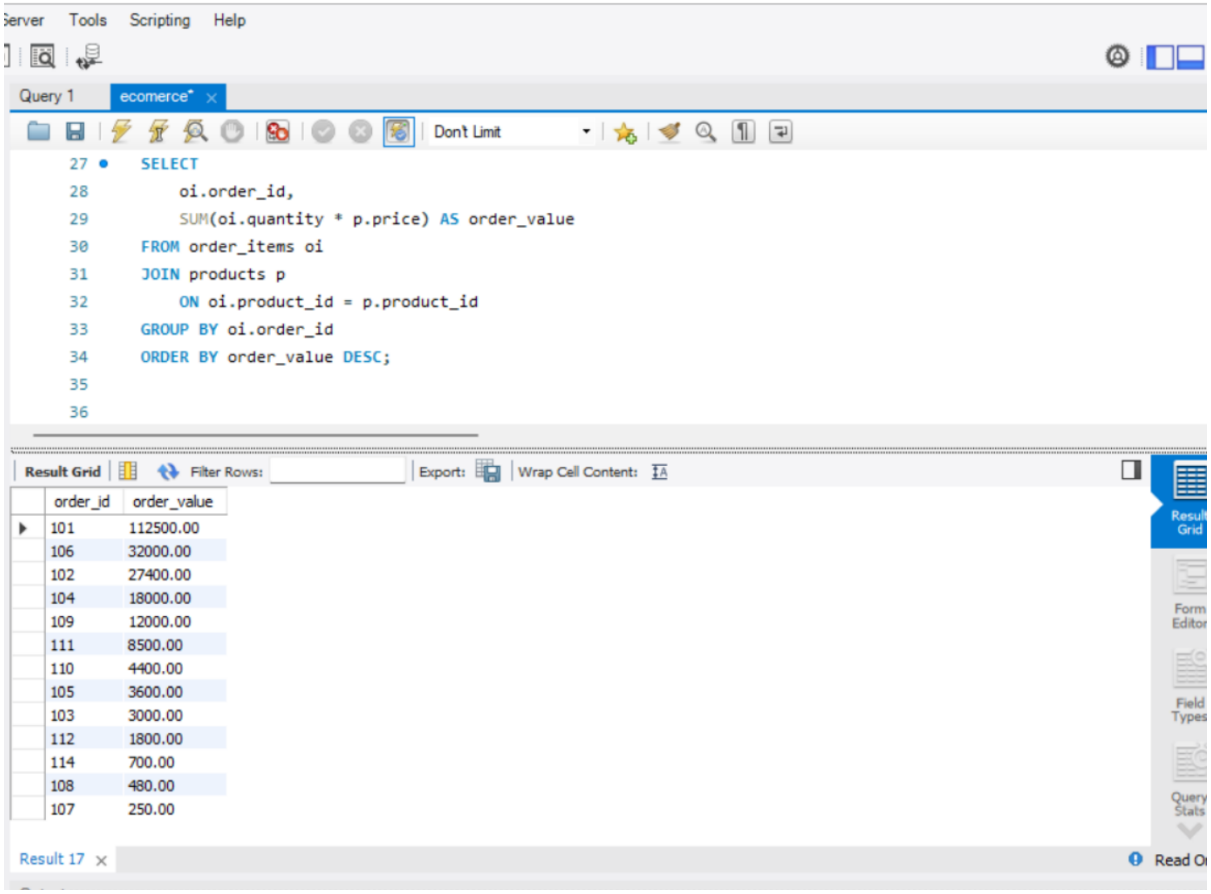
SQL Query 02 — Display complete order details



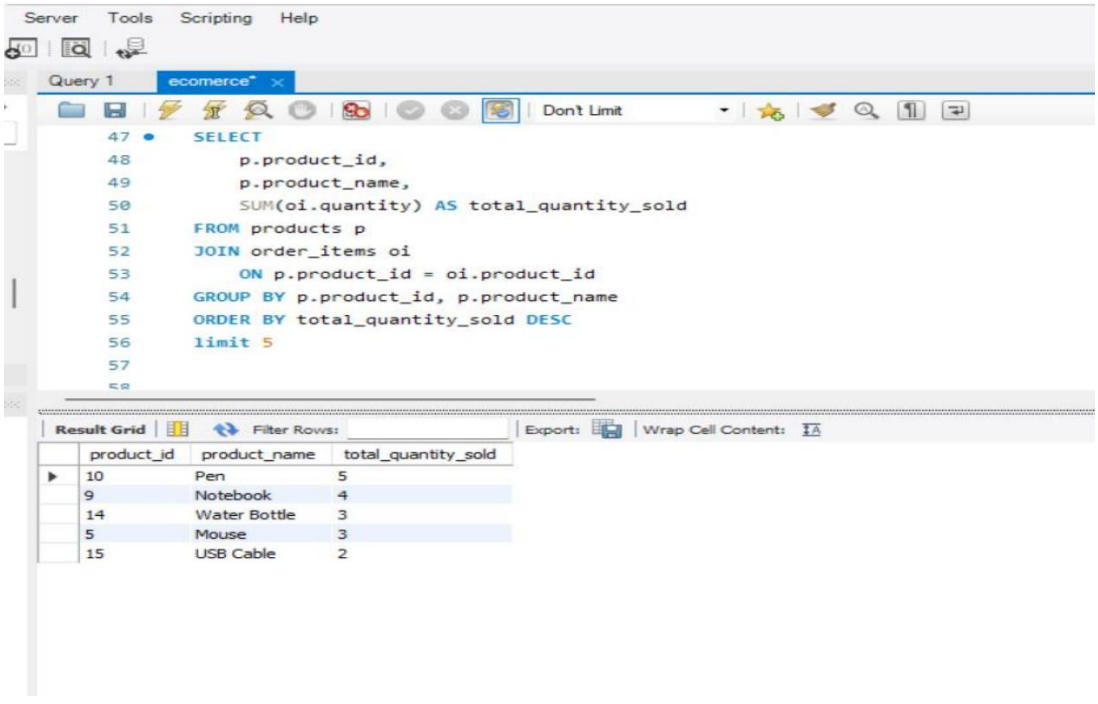
```
SELECT
    o.order_id,
    c.cus_name,
    c.city,
    o.order_date,
    o.order_status,
    p.payment_amount,
    p.payment_status
FROM order_details o
JOIN customer c
    ON o.customer_id = c.cus_id
LEFT JOIN payment_details p
    ON o.order_id = p.order_id
ORDER BY o.order_date DESC
limit 5;
```

order_id	cus_name	city	order_date	order_status	payment_amount	payment_status
115	Robert Jain	Indore	2026-03-25	Shipped	350.00	Success
114	Rohita Joseph	Chennai	2026-03-20	Delivered	600.00	Success
113	Anjali Kumari Gupta	Patna	2026-03-15	Cancelled	8500.00	Failure
112	Kavya Menon	Bangalore	2026-03-08	Pending	2200.00	Success
111	John Das	Bhubaneswar	2026-03-03	Delivered	12000.00	Refund

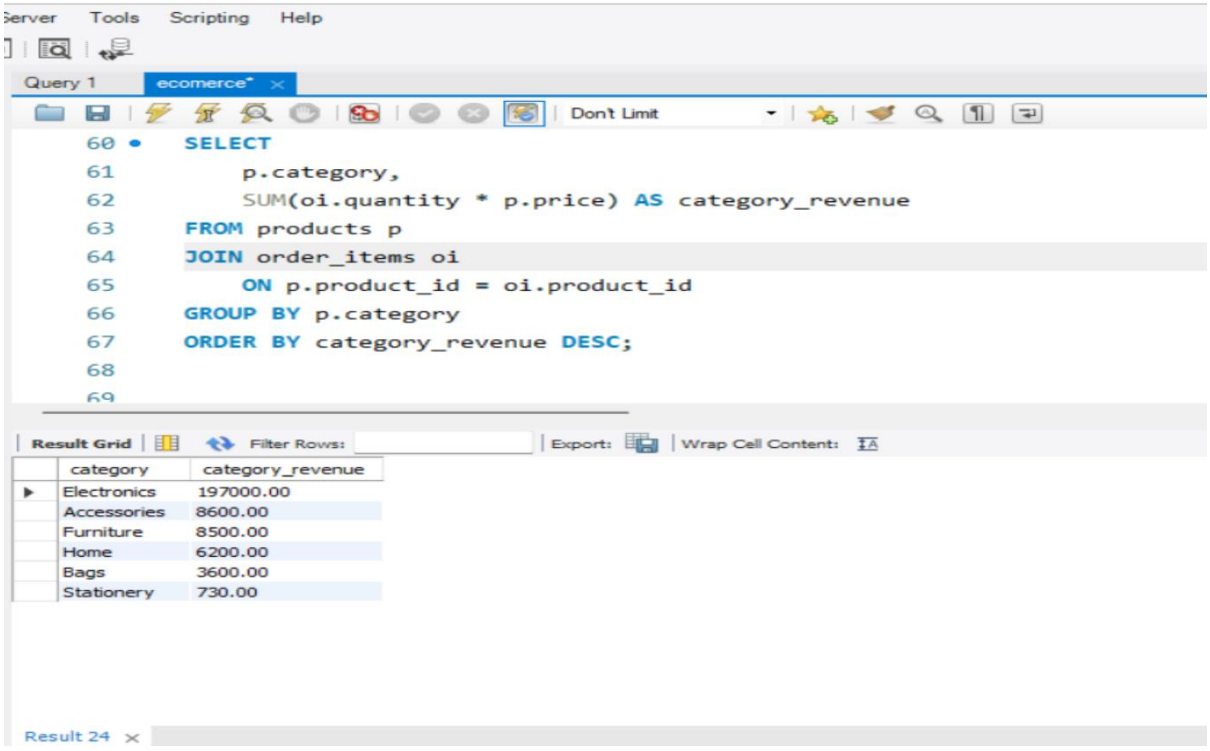
SQL Query 03 — Calculate the value of each order



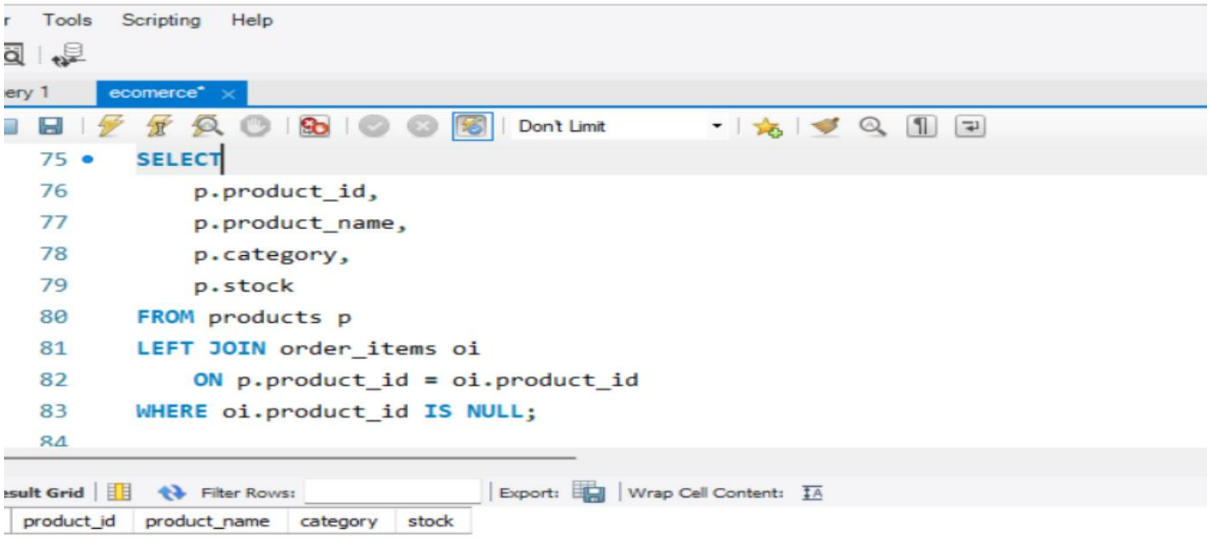
SQL Query 04 — Find total quantity sold for each product



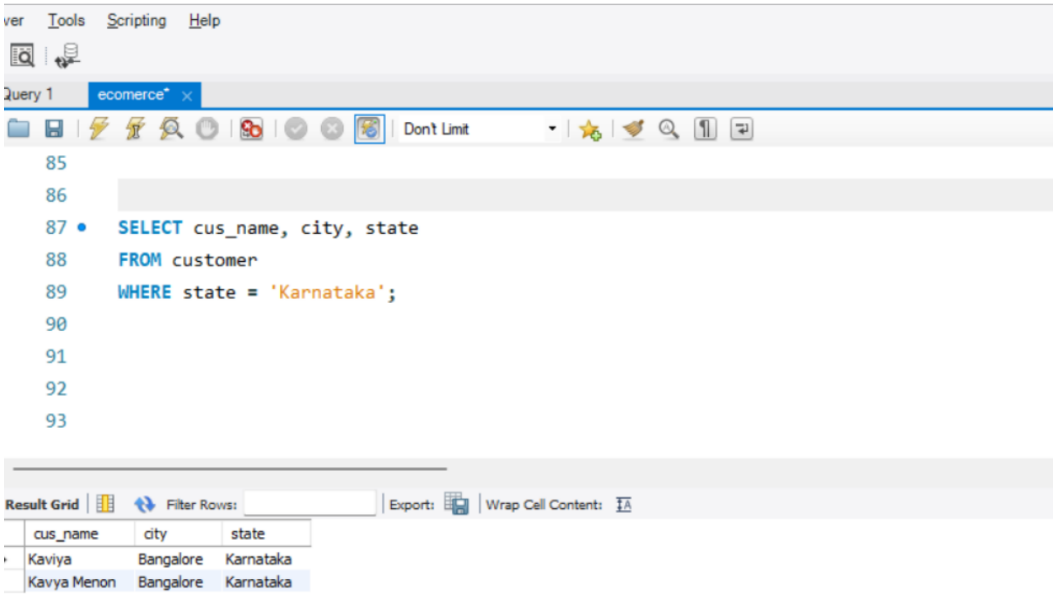
SQL Query 05 — Calculate revenue by product category



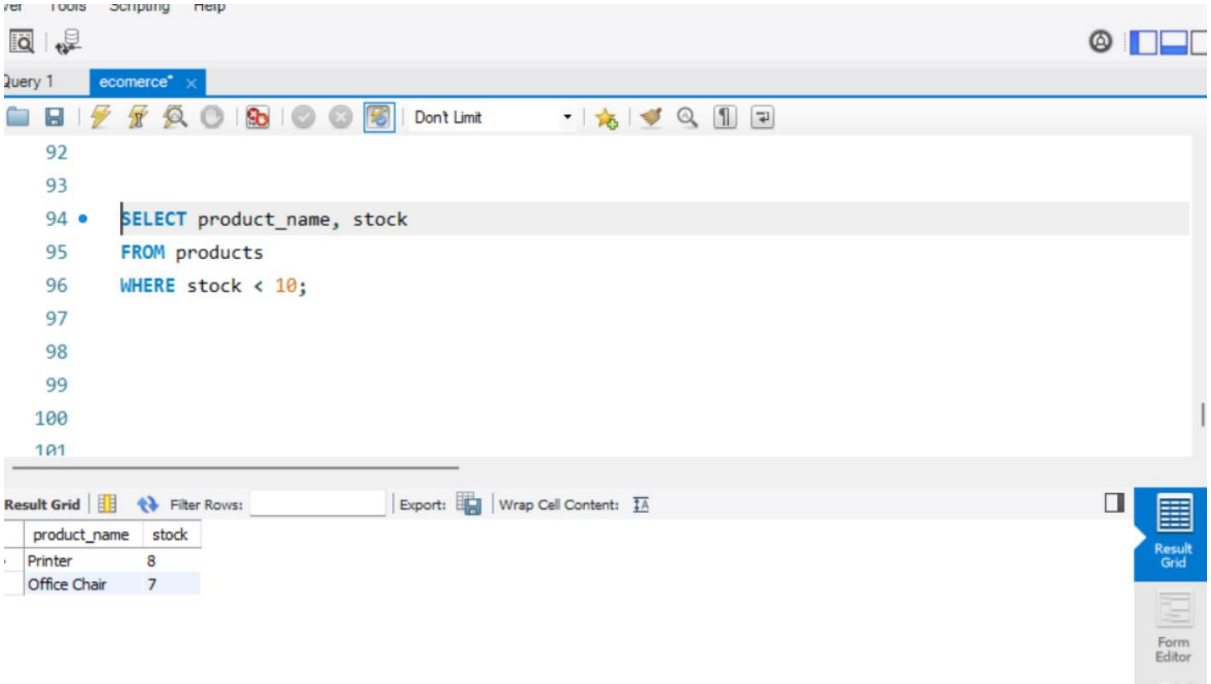
SQL Query 06 — Find products that have never been ordered



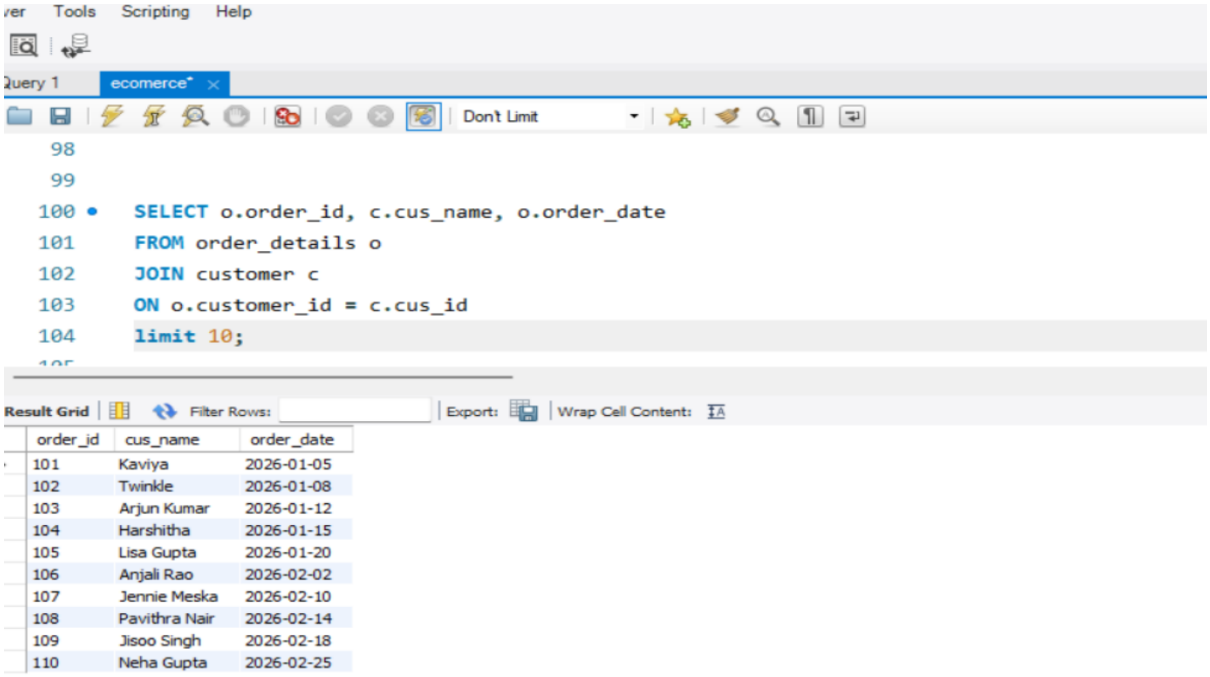
SQL Query 07 — Find customers from a particular state



SQL Query 08 — Find products with low stock



SQL Query 09 — Display orders with customer names

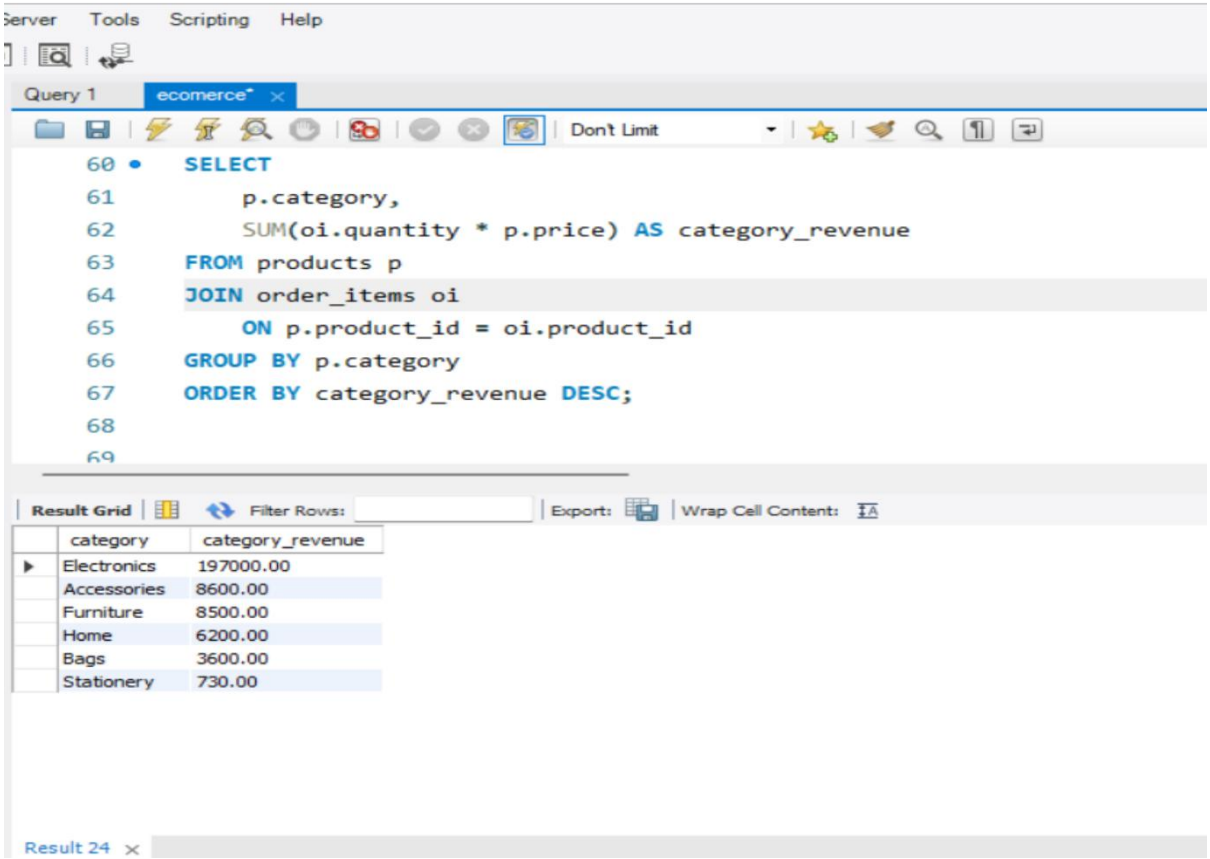


```
98
99
100 •  SELECT o.order_id, c.cus_name, o.order_date
101    FROM order_details o
102    JOIN customer c
103    ON o.customer_id = c.cus_id
104    limit 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	cus_name	order_date
101	Kaviya	2026-01-05
102	Twinkle	2026-01-08
103	Arjun Kumar	2026-01-12
104	Harshitha	2026-01-15
105	Lisa Gupta	2026-01-20
106	Anjali Rao	2026-02-02
107	Jennie Meska	2026-02-10
108	Pavithra Nair	2026-02-14
109	Jisoo Singh	2026-02-18
110	Neha Gupta	2026-02-25

SQL Query 10 — Find total quantity sold for each product



```
60 •  SELECT
61        p.category,
62        SUM(oi.quantity * p.price) AS category_revenue
63    FROM products p
64    JOIN order_items oi
65        ON p.product_id = oi.product_id
66    GROUP BY p.category
67    ORDER BY category_revenue DESC;
68
69
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	category_revenue
Electronics	197000.00
Accessories	8600.00
Furniture	8500.00
Home	6200.00
Bags	3600.00
Stationery	730.00

Result 24

SQL Query 11 — Identify products with stock below the average

```sql
SELECT
    product_id,
    product_name,
    stock
FROM products
WHERE stock < (
    SELECT AVG(stock)
    FROM products
)
ORDER BY stock ASC;
```

product_id	product_name	stock
13	Office Chair	7
11	Printer	8
1	Laptop	10
7	Tablet	12
6	Monitor	15
12	Desk Lamp	20
2	Smartphone	25
4	Keyboard	30
8	Backpack	35
3	Headphones	40

SQL Query 12 — Calculate total payment amount by payment method

```sql
SELECT
    payment_method,
    COUNT(*) AS total_transactions,
    SUM(payment_amount) AS total_payment_amount,
    AVG(payment_amount) AS average_payment
FROM payment_details
GROUP BY payment_method
ORDER BY total_payment_amount DESC;
```

payment_method	total_transactions	total_payment_amount	average_payment
Credit Card	4	84000.00	21000.000000
UPI	5	61350.00	12270.000000
Cash on Delivery	3	18720.00	6240.000000
Debit Card	3	16000.00	5333.333333

SQL Query 13 — Rank products by revenue

```
187
188 • WITH ProductRevenue AS (SELECT
189         p.product_id,
190         p.product_name,
191         SUM(oi.quantity * p.price) AS revenue
192     FROM products p
193     JOIN order_items oi
194         ON p.product_id = oi.product_id
195     GROUP BY p.product_id, p.product_name
196 )
197 SELECT product_id, product_name, revenue,
198         DENSE_RANK() OVER (
199         ORDER BY revenue DESC
200     ) AS revenue_rank
201 FROM ProductRevenue;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	revenue	revenue_rank
1	Laptop	110000.00	1
7	Tablet	32000.00	2
2	Smartphone	25000.00	3
6	Monitor	18000.00	4
11	Printer	12000.00	5
13	Office Chair	8500.00	6
12	Desk Lamp	4400.00	7
8	Backpack	3600.00	8

Result 44 | Output

SQL Query 14 — Find the highest-priced product in each category

```
202
203 • WITH RankedProducts AS (SELECT product_id,product_name,category,price,
204             RANK() OVER (
205             PARTITION BY category
206             ORDER BY price DESC
207         ) AS price_rank
208     FROM products
209 )
210 SELECT *
211 FROM RankedProducts
212 WHERE price_rank = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	revenue	revenue_rank
1	Laptop	110000.00	1
7	Tablet	32000.00	2
2	Smartphone	25000.00	3
6	Monitor	18000.00	4
11	Printer	12000.00	5
13	Office Chair	8500.00	6
12	Desk Lamp	4400.00	7
8	Backpack	3600.00	8
4	Keyboard	3000.00	9
3	Headphones	2500.00	10
5	Mouse	2400.00	11
14	Water Bottle	1800.00	12

Result 44 | Output

SQL Query 15 — Find the top three products by revenue

```
215 •   SELECT
216         p.product_id,
217         p.product_name,
218         p.category,
219         SUM(oi.quantity) AS units_sold,
220         SUM(oi.quantity * p.price) AS revenue
221     FROM products p
222     JOIN order_items oi
223         ON p.product_id = oi.product_id
224     GROUP BY p.product_id, p.product_name, p.category
225     ORDER BY revenue DESC
226     LIMIT 3;
```

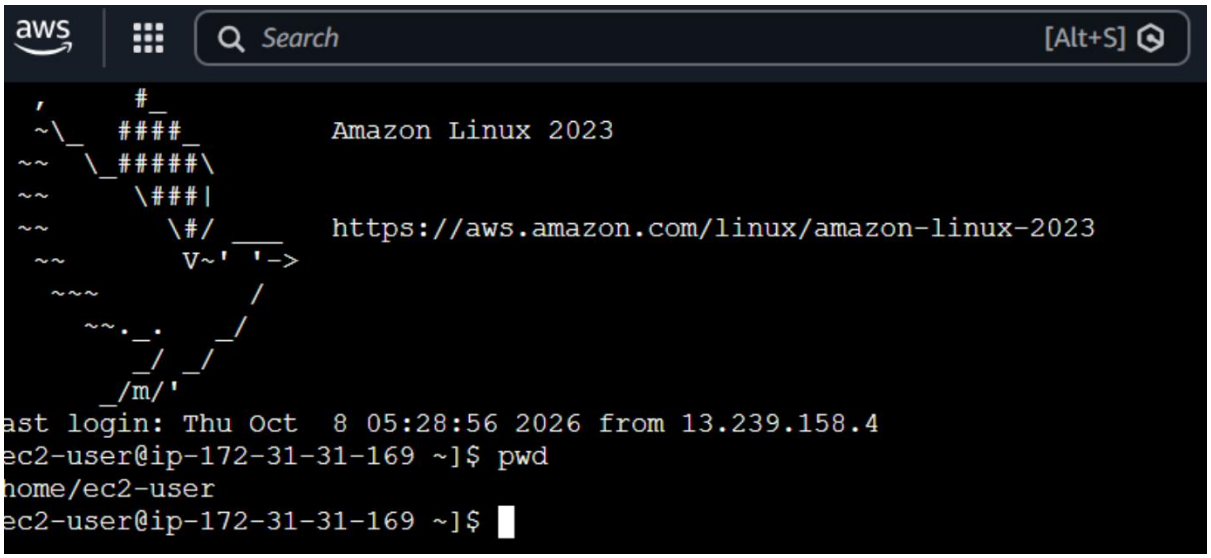


product_id	product_name	category	units_sold	revenue
1	Laptop	Electronics	2	110000.00
7	Tablet	Electronics	1	32000.00
2	Smartphone	Electronics	1	25000.00

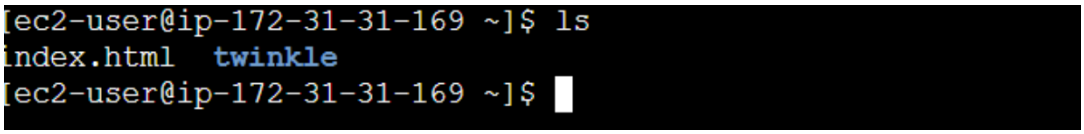
Linux Practice

Amazon EC2 is the AWS service used to create and run virtual servers in the cloud. The Linux commands in this section were run in the terminal of an Amazon EC2 instance running Amazon Linux 2023, logged in as ec2-user. Each command is shown with its terminal output.

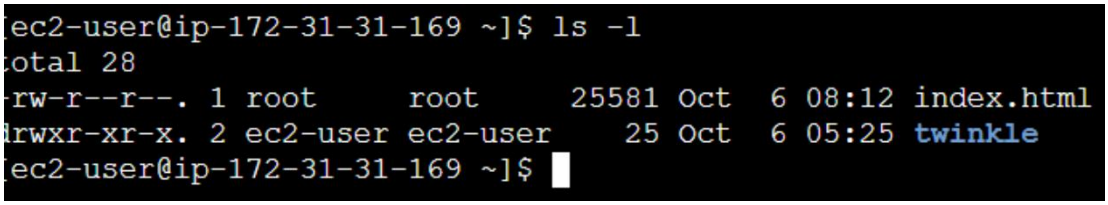
Linux Command 01 — pwd — Show the current directory



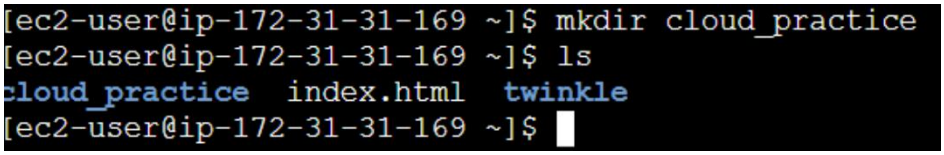
Linux Command 02 — ls — List files and folders



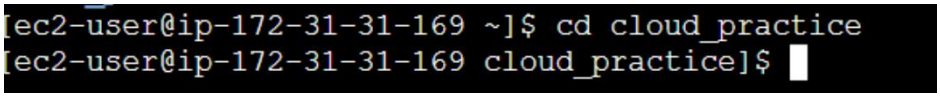
Linux Command 03 — ls -l — Show detailed file information



Linux Command 04 — mkdir — Create a directory



Linux Command 05 — cd — Enter a directory



Linux Command 06 — touch — Create an empty file

```
[ec2-user@ip-172-31-31-169 cloud_practice]$ touch employees.txt
[ec2-user@ip-172-31-31-169 cloud_practice]$ ls
employees.txt
[ec2-user@ip-172-31-31-169 cloud_practice]$
```

Linux Command 07 — echo — Write text to a file

```
[ec2-user@ip-172-31-31-169 cloud_practice]$ echo "employee data" > employees.txt
[ec2-user@ip-172-31-31-169 cloud_practice]$
```

Linux Command 08 — cat — Display file contents

```
[ec2-user@ip-172-31-31-169 cloud_practice]$ cat employees.txt
employee data
[ec2-user@ip-172-31-31-169 cloud_practice]$
```

Linux Command 09 — cp — Copy a file

```
[ec2-user@ip-172-31-31-169 cloud_practice]$ cp employees.txt backup.txt
[ec2-user@ip-172-31-31-169 cloud_practice]$ ls
backup.txt  empl  employees.txt  employees.txtecho
[ec2-user@ip-172-31-31-169 cloud_practice]$
```

Linux Command 10 — mv — Move a file or folder, or rename it

```
[ec2-user@ip-172-31-31-169 employee]$ mv backup.txt test.txt
[ec2-user@ip-172-31-31-169 employee]$ ls
test.txt
```

Linux Command 11 — ps — View running processes

Shows the processes currently running on the system.

```
[ec2-user@ip-172-31-31-169 employee]$ ps
    PID TTY          TIME CMD
 275927 pts/0    00:00:00 bash
 276936 pts/0    00:00:00 ps
[ec2-user@ip-172-31-31-169 employee]$
```

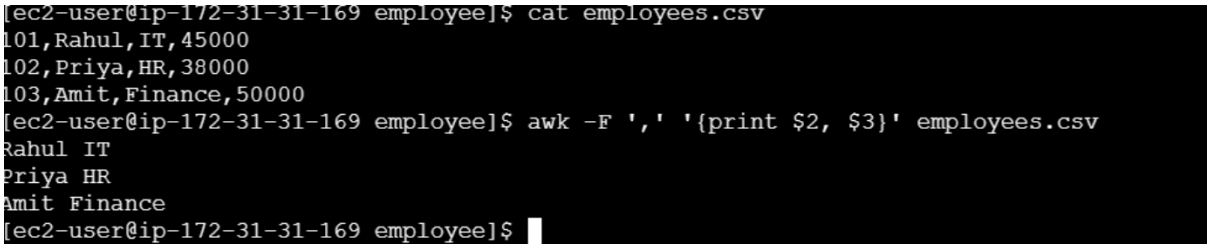
Linux Command 12 — chmod — Change file permissions

Create a sample file and change its permissions.

```
[ec2-user@ip-172-31-31-169 employee]$ touch report.txt
[ec2-user@ip-172-31-31-169 employee]$ chmod 640 report.txt
[ec2-user@ip-172-31-31-169 employee]$ ls -l report.txt
-rw-r-----. 1 ec2-user ec2-user 0 Oct 11 20:01 report.txt
[ec2-user@ip-172-31-31-169 employee]$
```

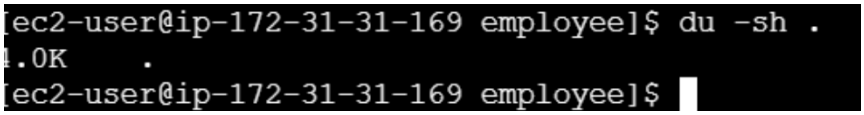
Linux Command 13 — awk — Extract selected columns from data

Useful for processing CSV-style data and logs.



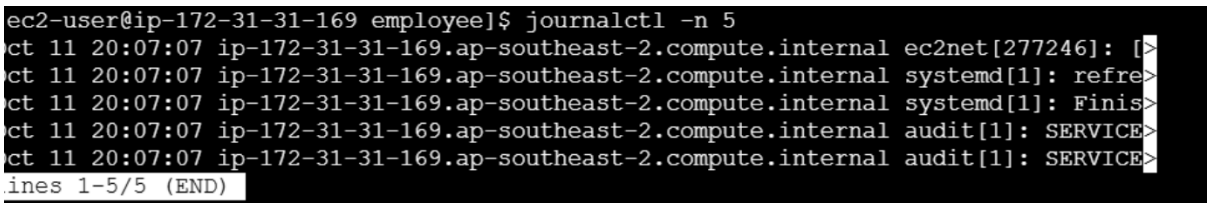
Linux Command 14 — du -sh — Check directory size

Shows the total size of a directory in a readable format.



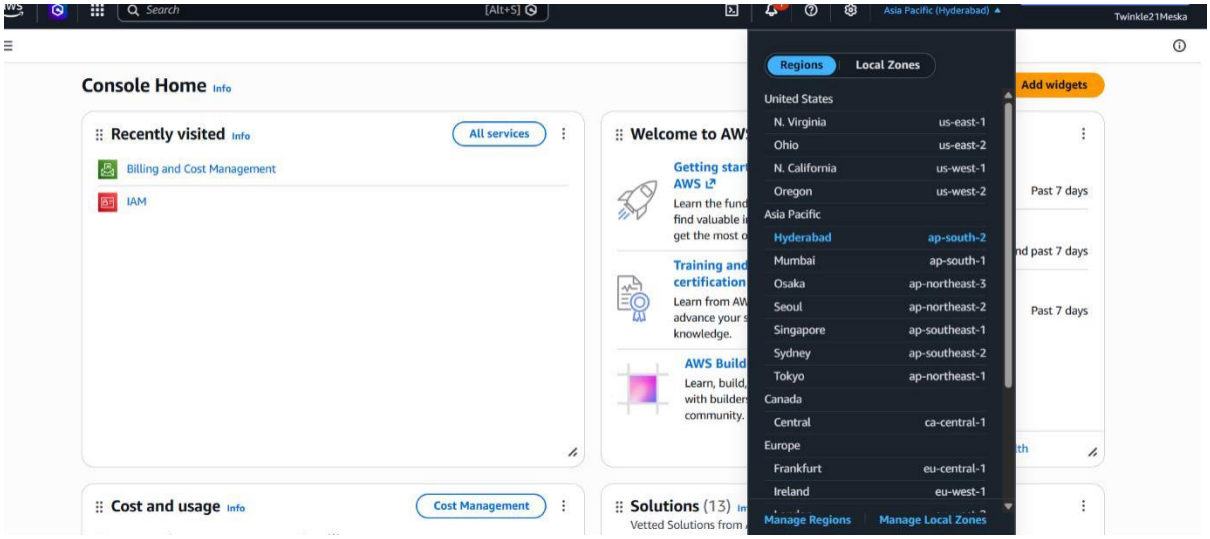
Linux Command 15 — journalctl — Inspect system logs

On systems using systemd, this command displays system logs.



AWS Account / Region

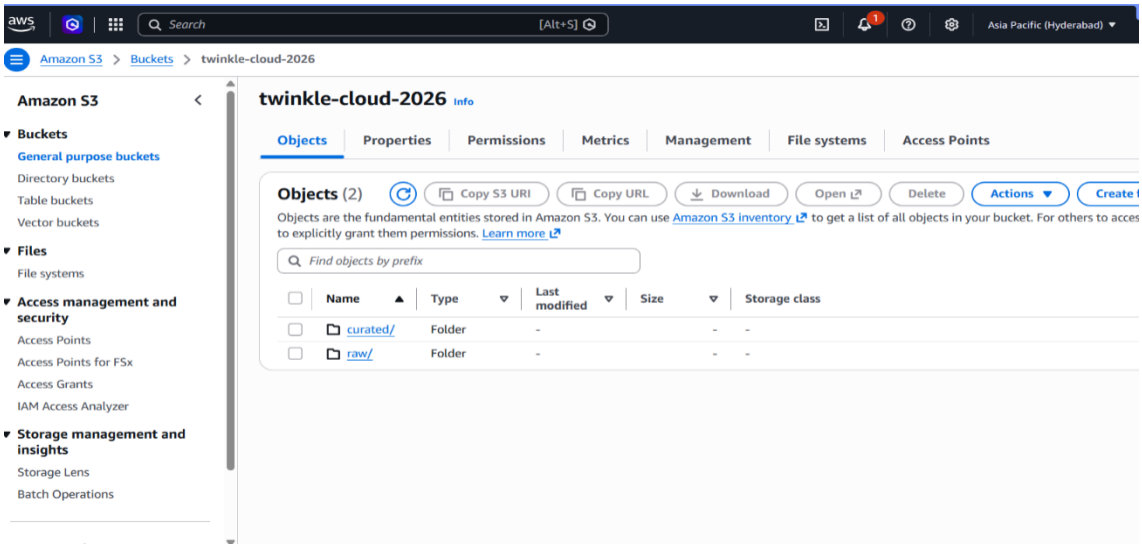
All AWS resources for this practical were created from the AWS Management Console in the region shown below.



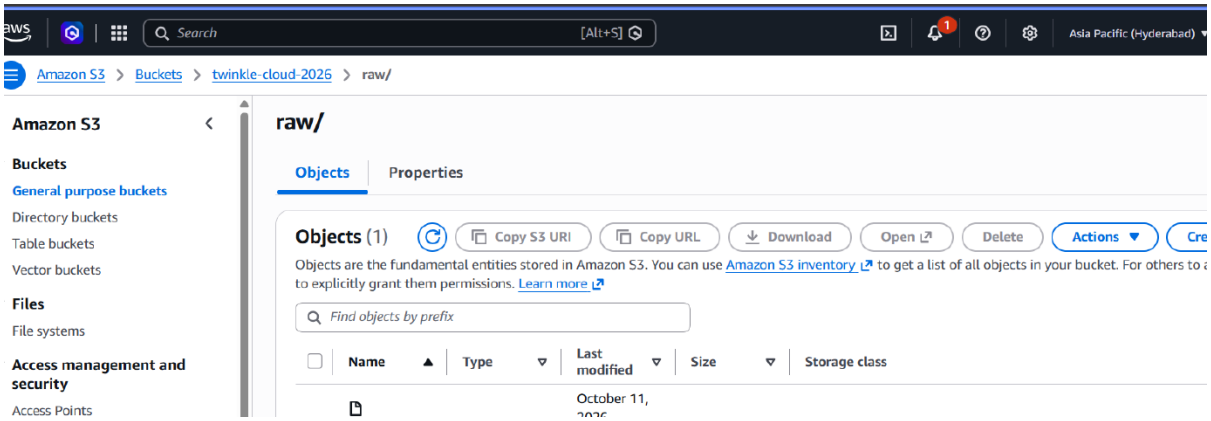
AWS Console Home with the selected region Asia Pacific (Hyderabad), ap-south-2.

Amazon S3

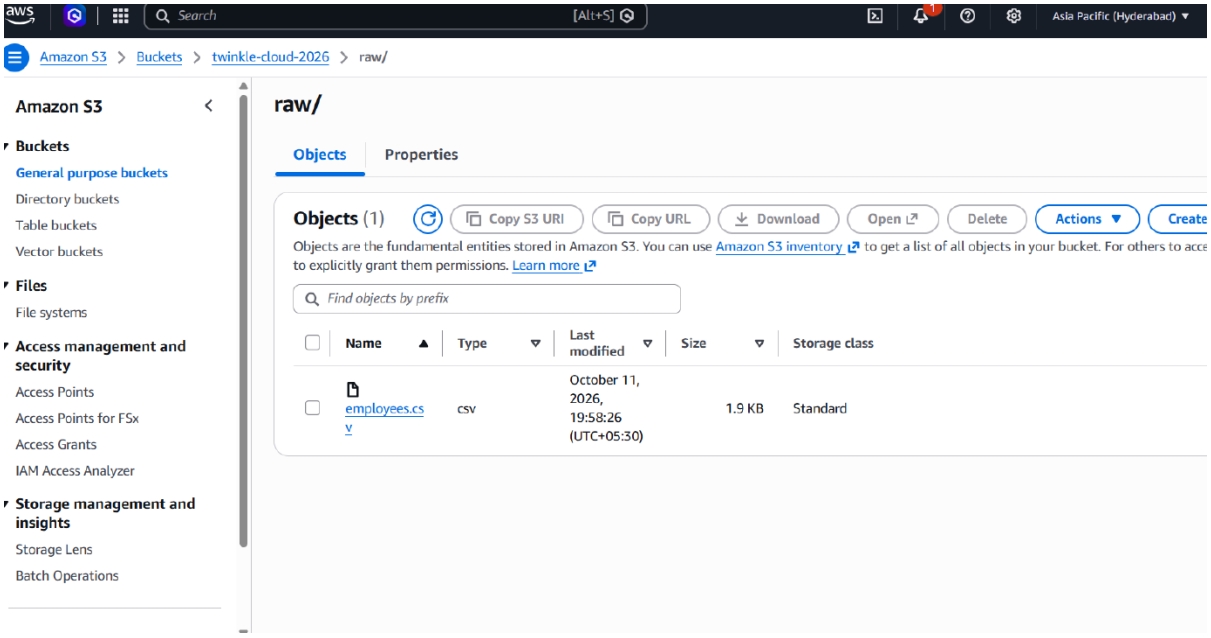
An S3 bucket was created with two folders: raw/ for the original data and curated/ for the transformed output. The file employees.csv was uploaded inside raw/.



S3 bucket twinkle-cloud-2026 containing the raw/ and curated/ folders.



The raw/ folder inside the bucket.

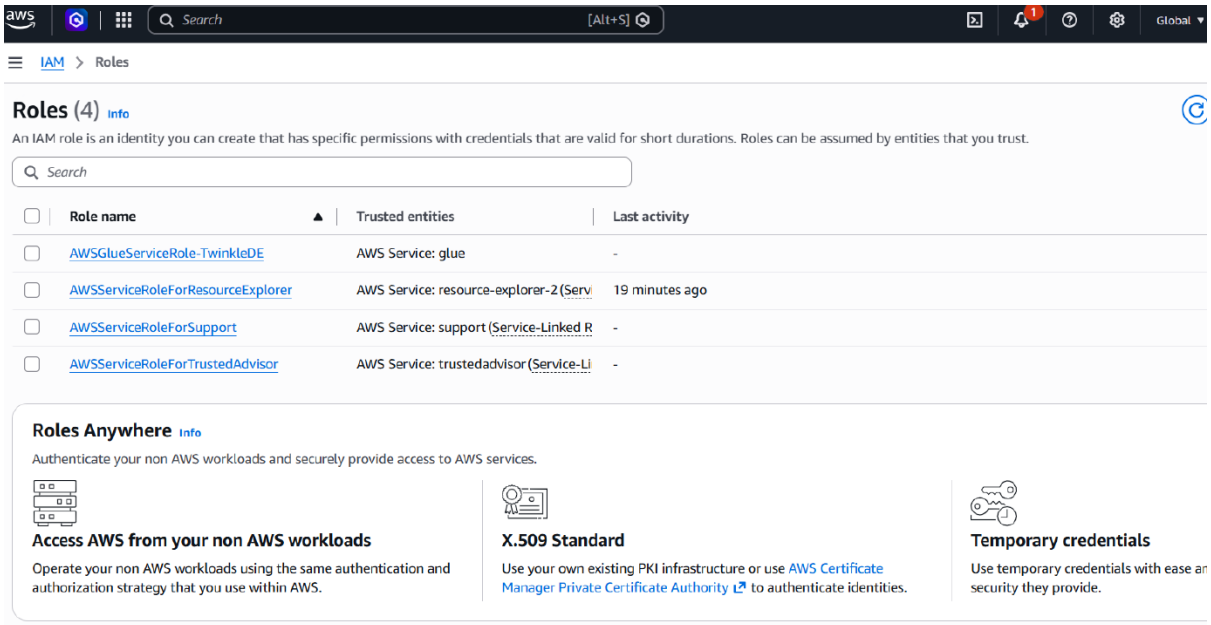


employees.csv (CSV, 1.9 KB) uploaded inside the raw/ folder.

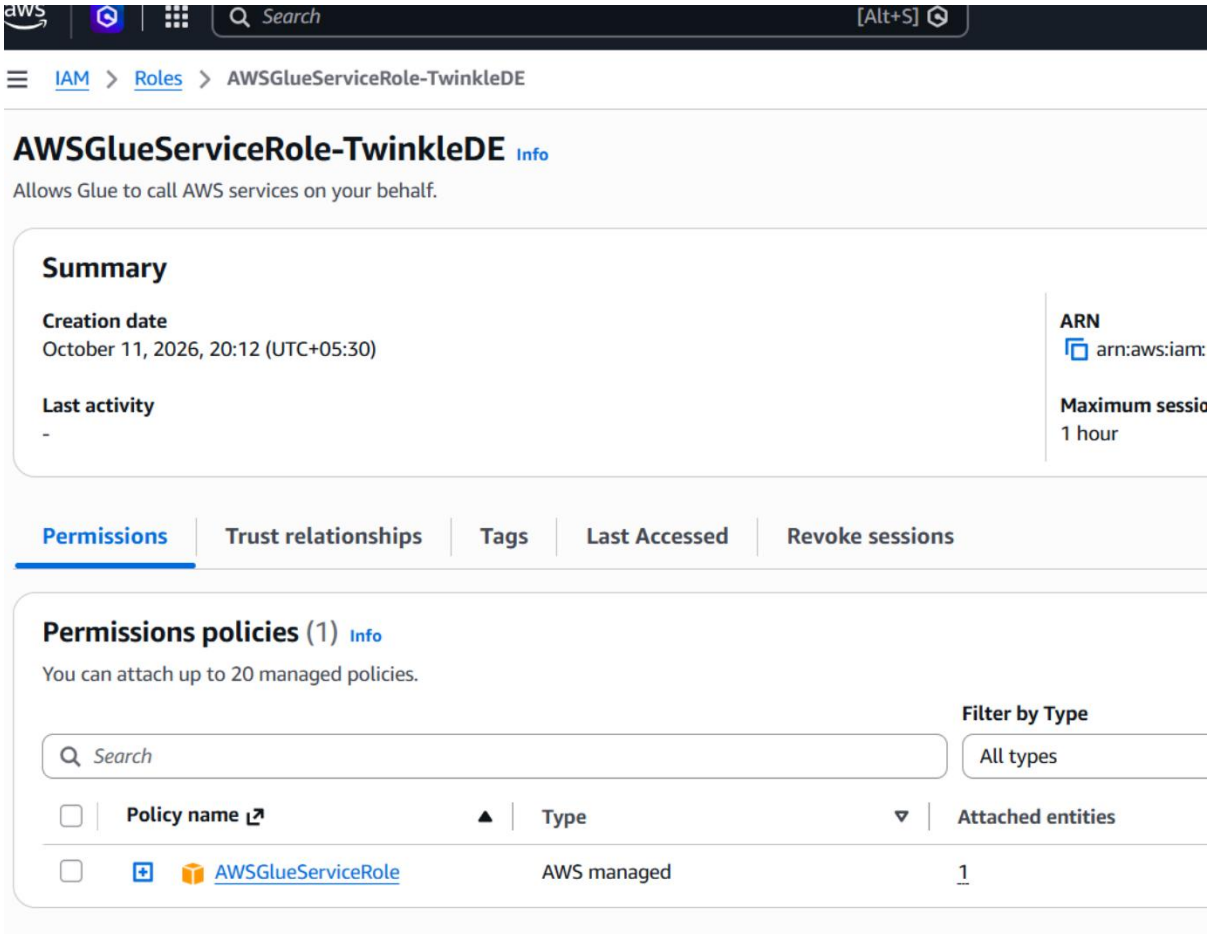
IAM

Glue needs an IAM role to read the data from S3 and to write the results back. The role AWSGlueServiceRole-TwinkleDE was created with Glue as the trusted service. It has the AWS managed policy AWSGlueServiceRole and a custom inline policy,

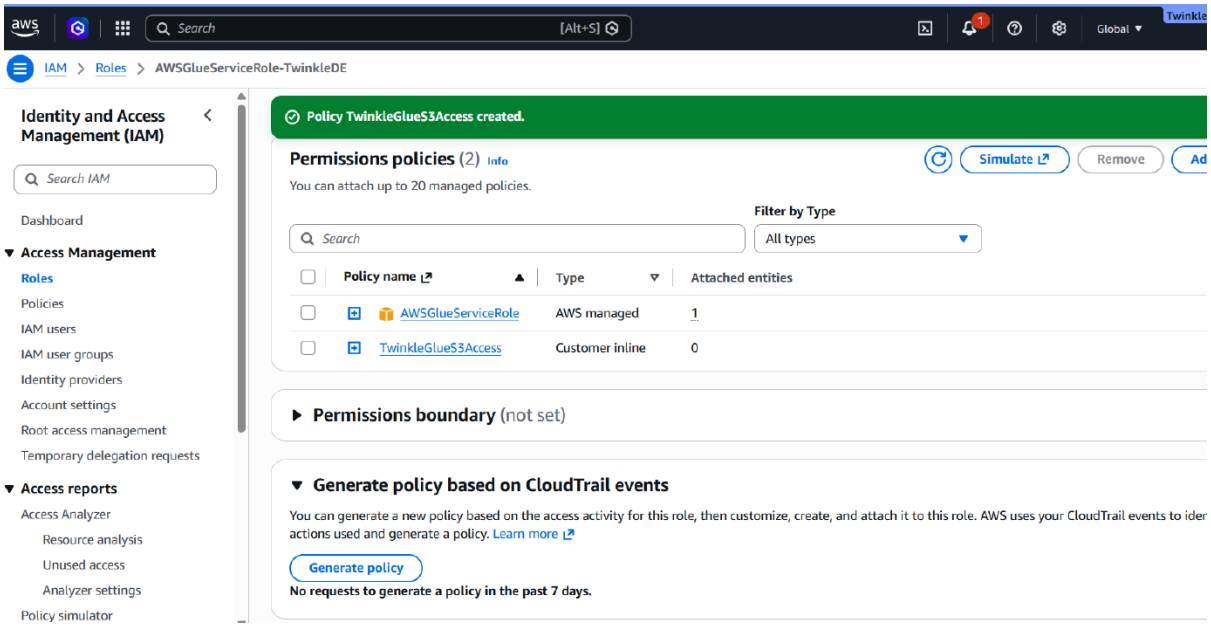
TwinkleGlueS3Access, that allows access to the S3 bucket. No passwords or access keys are shown in the screenshots.



IAM Roles page showing the role used for AWS Glue: AWSGlueServiceRole-TwinkleDE (trusted entity: AWS Service: glue).



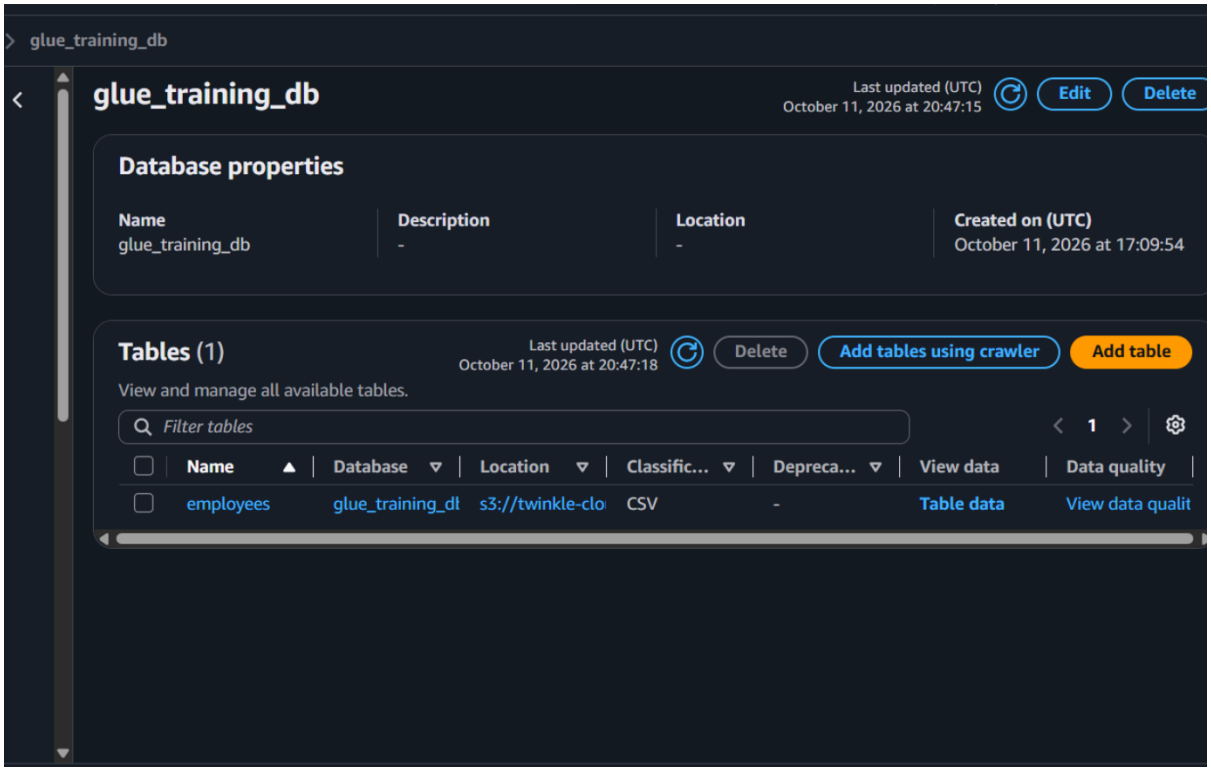
Role summary with the AWS managed policy AWSGlueServiceRole attached.



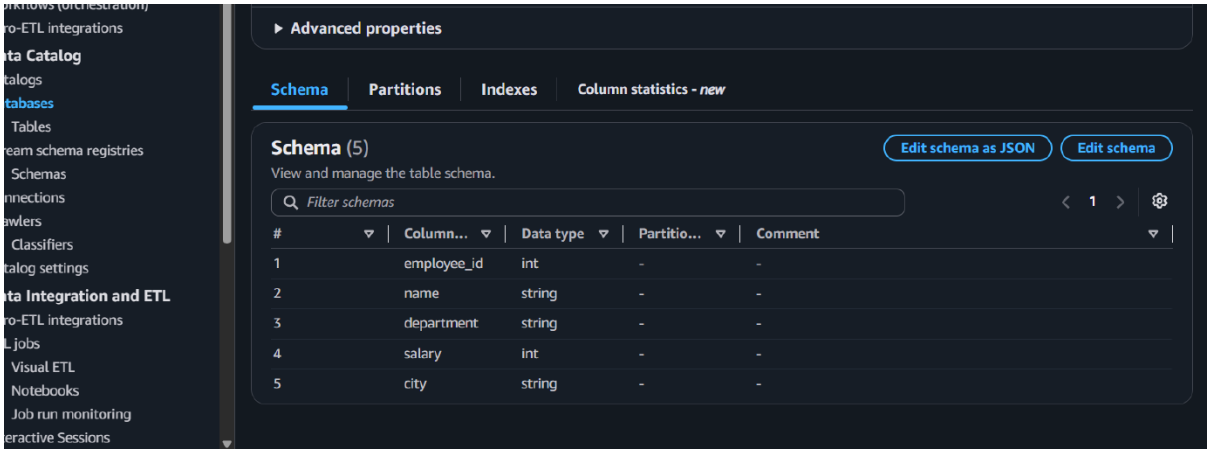
Custom inline policy TwinkleGlueS3Access created and attached to the role.

AWS Glue Database

A database named glue_training_db was created in the AWS Glue Data Catalog to hold the table discovered by the crawler.



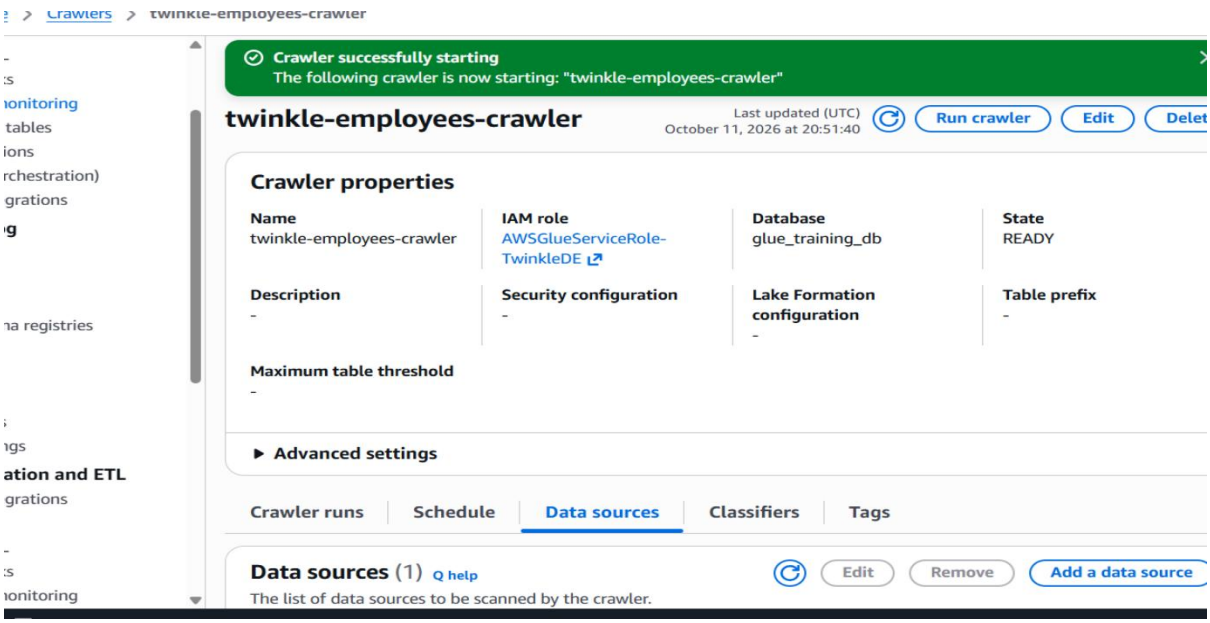
Database glue_training_db with its properties and the employees table.

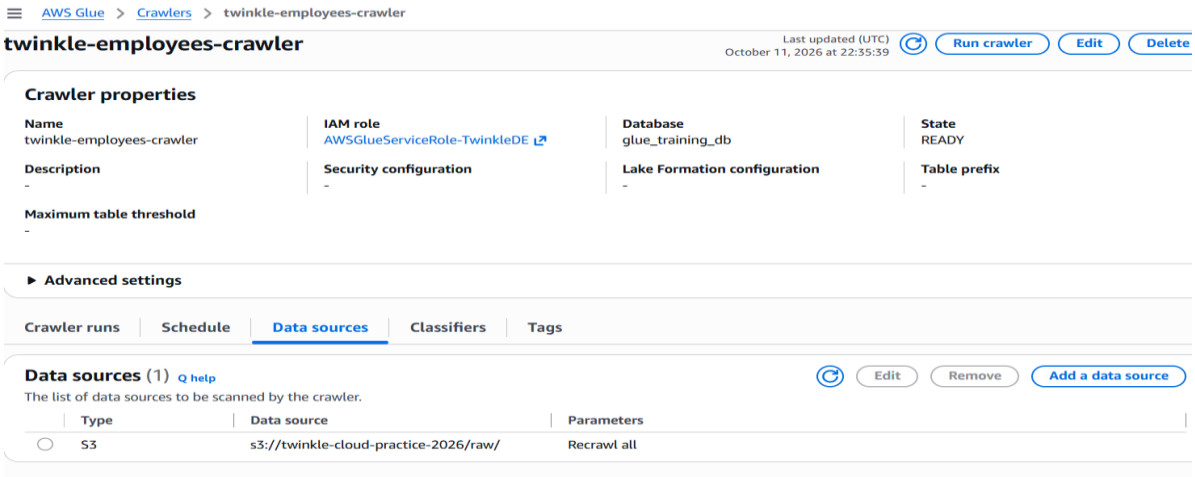


Schema of the employees table: employee_id, name, department, salary and city.

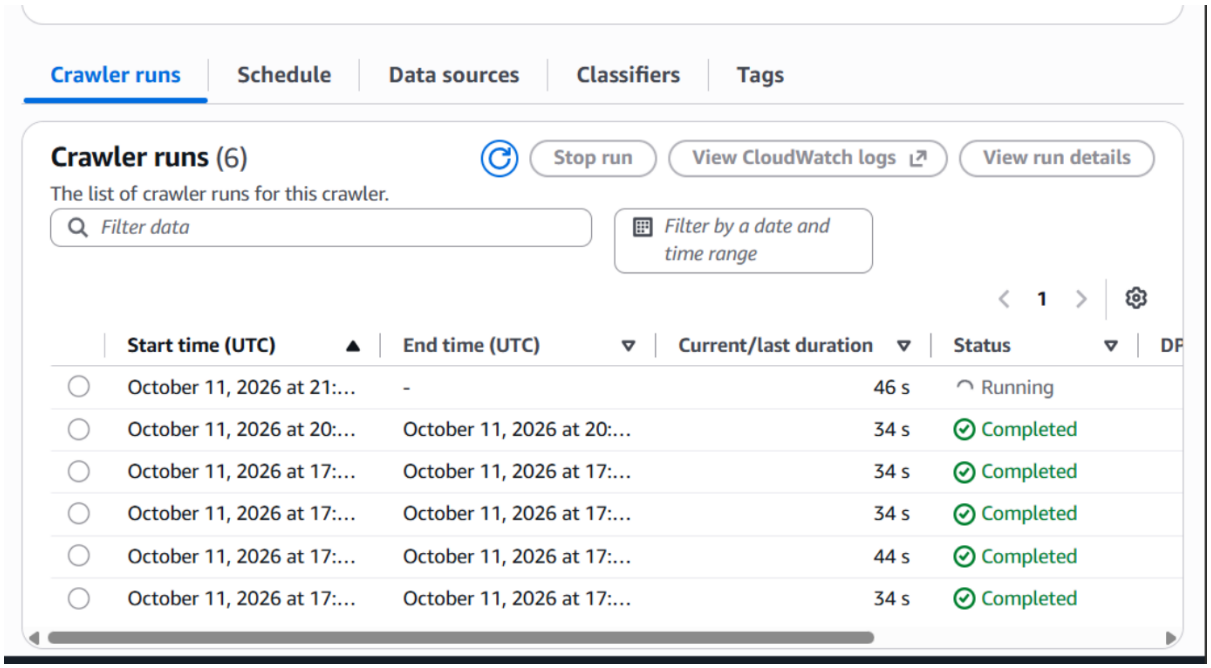
Glue Crawler

The crawler twinkle-employees-crawler scans the raw/ folder, reads the CSV file and creates a table in glue_training_db. It uses the IAM role AWSGlueServiceRole-TwinkleDE and runs on demand.





Crawler Execution



Crawler runs: earlier runs show Completed; the most recent run was still running when this screenshot was taken.

Glue Data Catalog

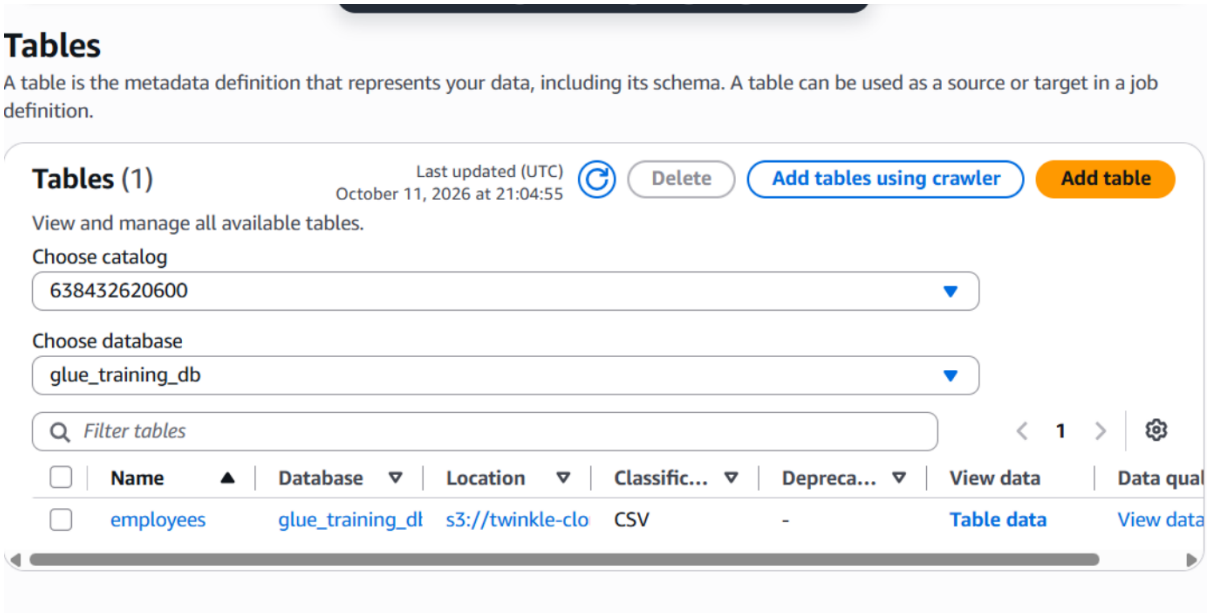


Table employees created in database glue_training_db (classification: CSV).

Table Schema

Schema | Partitions | Indexes | Column statistics - *new*

Schema (5)

View and manage the table schema.

Edit schema as JSON | Edit schema

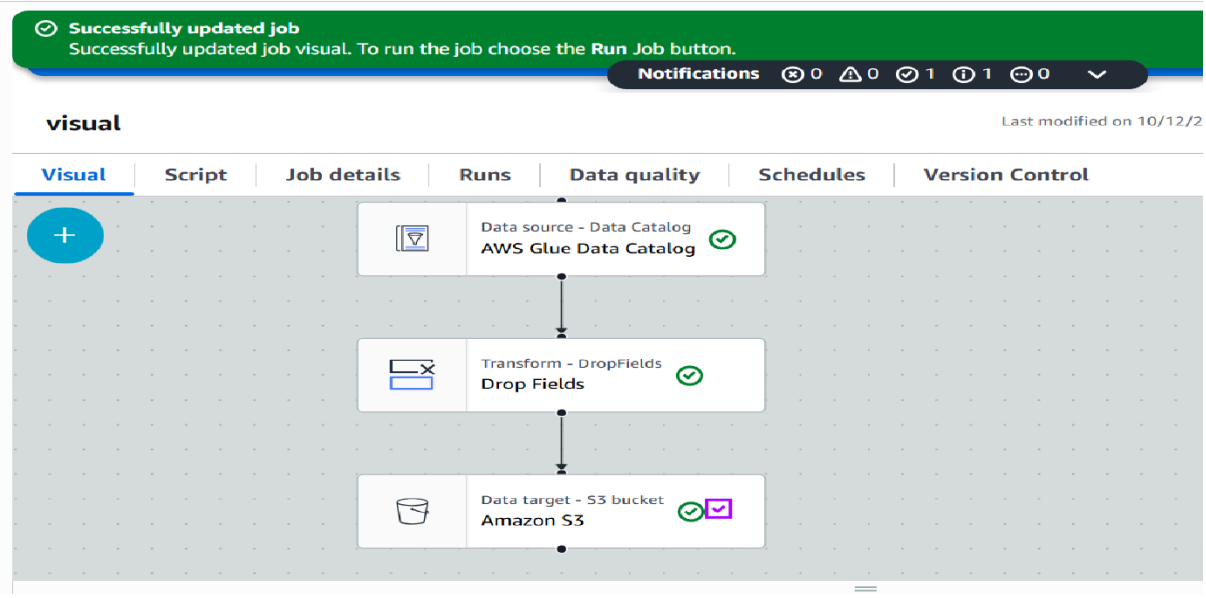
Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	employee_id	int	-	-
2	name	string	-	-
3	department	string	-	-
4	salary	int	-	-
5	city	string	-	-

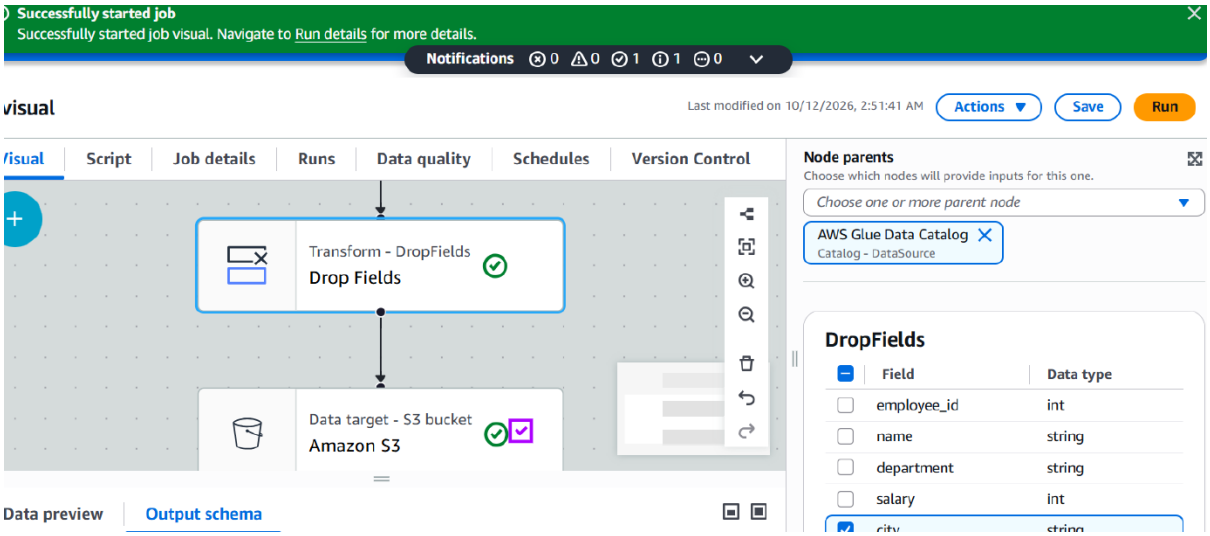
Table schema with data types: employee_id (int), name (string), department (string), salary (int), city (string).

Glue Visual ETL



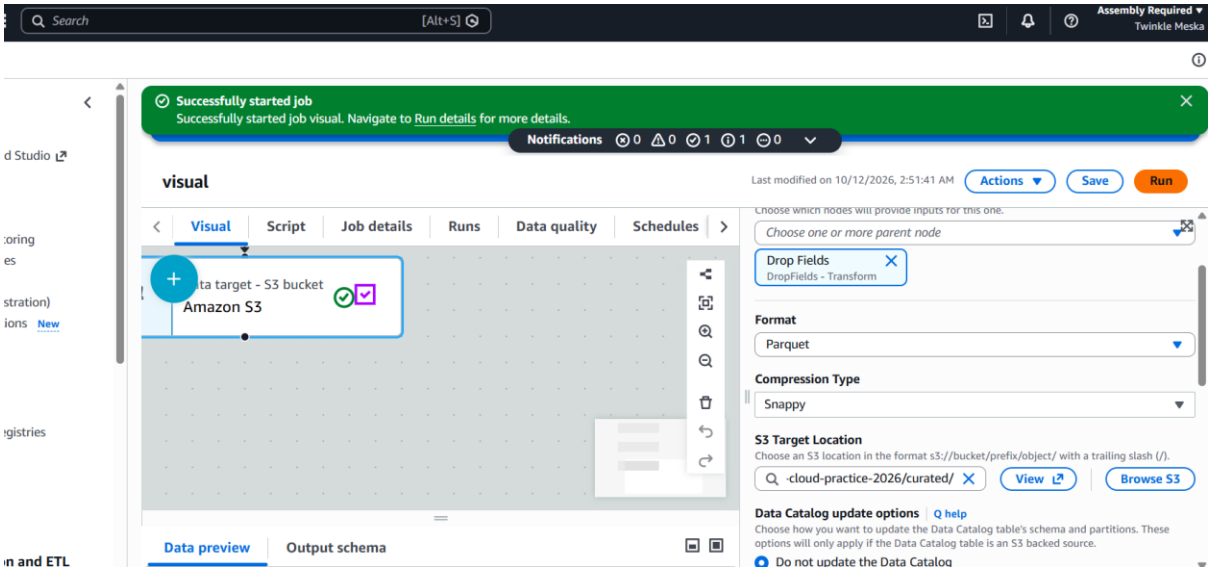
Visual ETL job "visual": AWS Glue Data Catalog source → Drop Fields transform → Amazon S3 target.

Glue Transformation

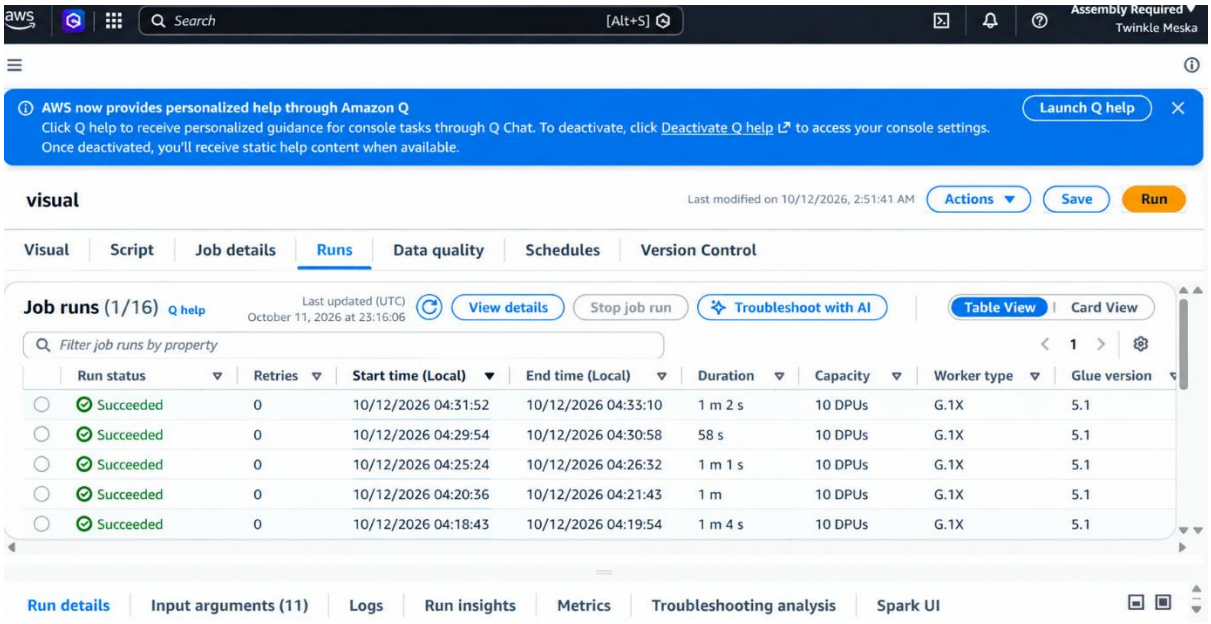


Drop Fields transformation: the city column is selected for removal. The remaining columns are employee_id, name, department and salary.

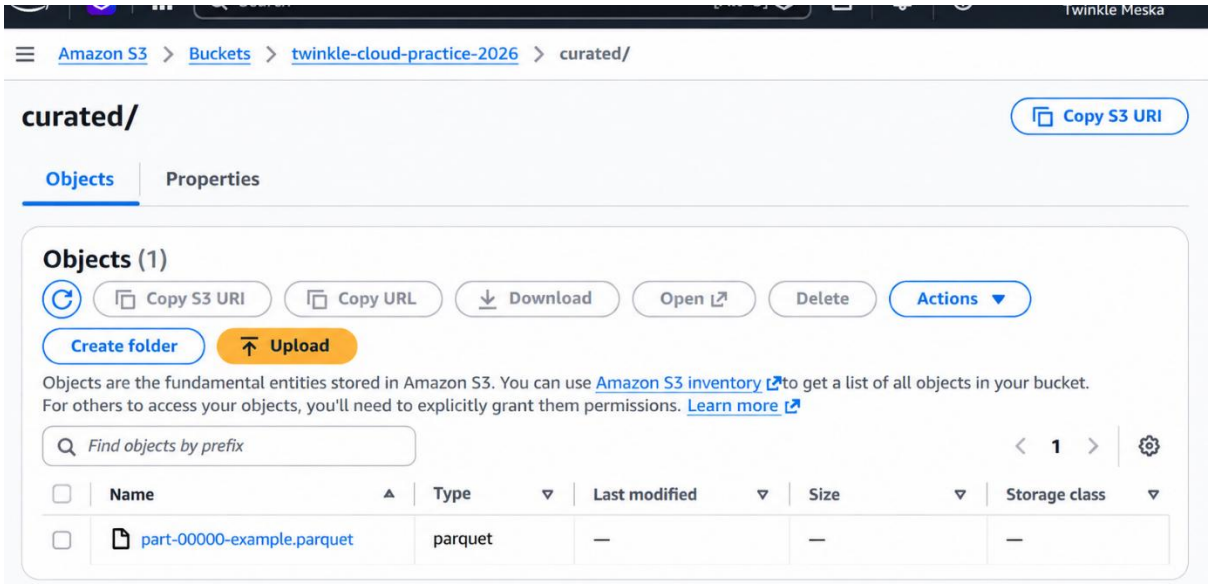
S3 Target Configuration



Glue ETL Job Execution



Final S3 Output



Final Explanation: AWS Glue ETL Workflow

1. Amazon S3

Amazon S3 is used to store the data in this workflow. I created an S3 bucket with two folders, raw and curated. The employee CSV file was uploaded into the raw folder. The raw folder stores the original data, while the curated folder is used to store the transformed data.

2. AWS Glue Crawler

The AWS Glue crawler scans the employee CSV file stored in the raw folder. It identifies the file structure, column names and data types. The crawler uses an IAM role to access the S3 data and stores the discovered metadata in the Glue Data Catalog.

3. AWS Glue Data Catalog

The AWS Glue Data Catalog stores metadata about the dataset rather than the actual employee records. I created a database named glue_training_db, and the crawler created an employees table containing the columns employee_id, name, department, salary and city. This table is used as the source for the Glue ETL job.

4. AWS Glue Visual ETL

AWS Glue Visual ETL allows me to create an ETL workflow using a visual interface instead of writing all the code manually. I created a job named visual and connected the Data Catalog source to a transformation node and an Amazon S3 target. The source reads the employee dataset through the Data Catalog table.

5. Transformation

I used the Drop Fields transformation to remove the city column because it is not required in the final dataset. The remaining columns are employee_id, name, department and salary. This transformation removes unwanted fields from the data before storing the output.

6. S3 Curated Data

The transformed data is stored in the curated folder of the S3 bucket in Parquet format. Parquet is a columnar file format commonly used for analytics. After the Glue job completes successfully with the correct S3 permissions and target configuration, the generated output files appear in the curated folder.

Workflow

S3 Raw Data → Glue Crawler → Glue Data Catalog → Glue Visual ETL → Drop Fields Transformation → S3 Curated Data